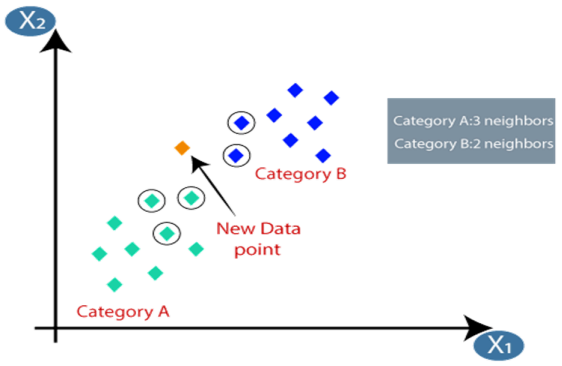
UNIT – III: Supervised Learning – II

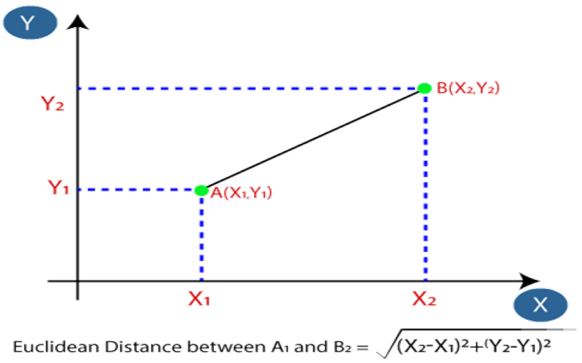
K-NN classifier – Logistic regression – Perceptron – Single layer & Multi-layer Support Vector Machines – Linear & Non-linear – Metrics & Error Correction.

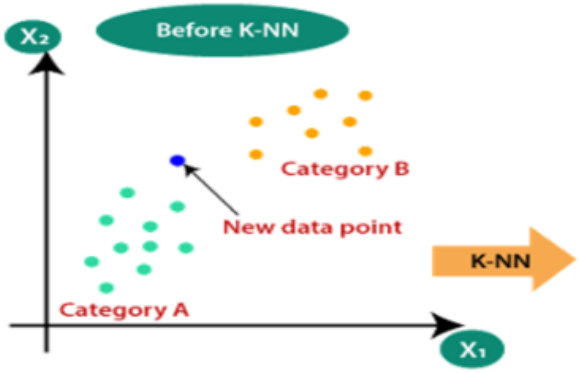
The **K-Nearest Neighbour (KNN) Classifier** is a simple and effective machine learning algorithm used primarily for classification tasks. It operates based on the idea that data points that are similar tend to have the same label.

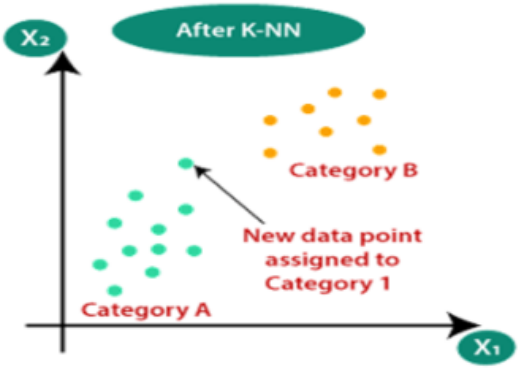
Key Concepts of KNN:

1. **Similarity Based on Distance**:
   - The KNN algorithm compares the distance between the input data point (the point you want to classify) and all the other points in the dataset.
   - It often uses **Euclidean distance** to measure how "close" data points are to each other.
2. **K Nearest Neighbours**:
   - After calculating the distance between the new point and all the points in the dataset, it identifies the **K closest points** (where **K** is a number chosen by the user).
   - The **most common class** among these K neighbors is then assigned to the new data point.
3. **Lazy Learning**:
   - KNN is considered a **lazy learner** because it doesn't actually learn a model during the training phase. Instead, it memorizes the entire dataset and only makes calculations when it needs to classify a new data point.



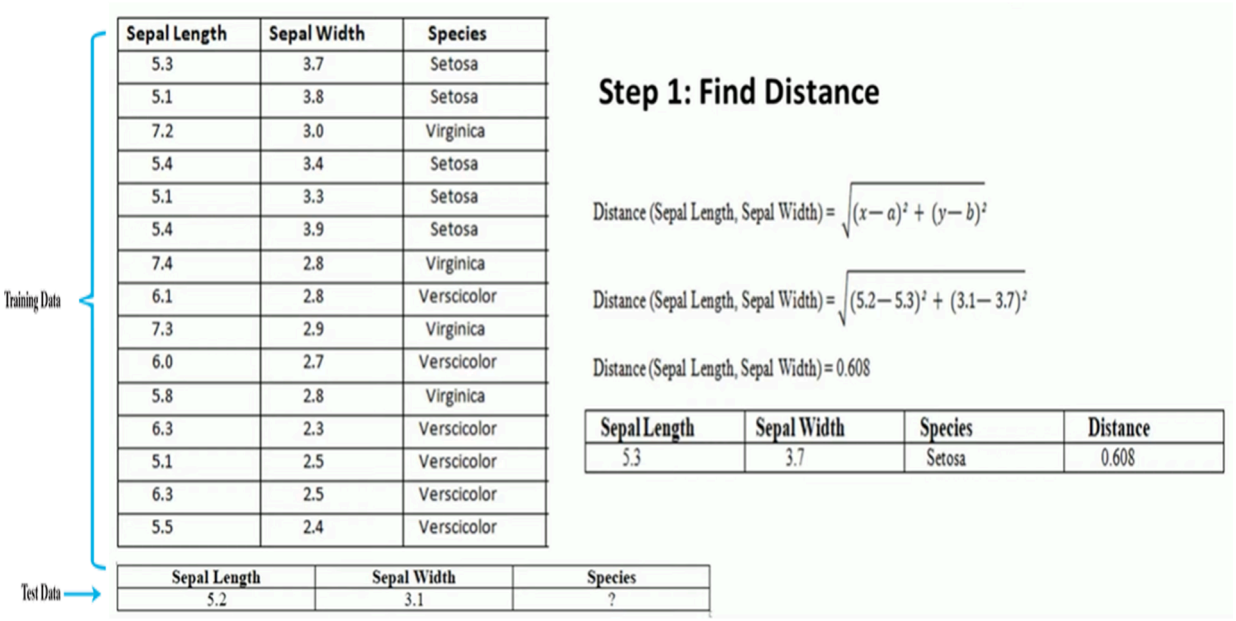






Choosing the value of k:

- **Square Root Rule**: A common way to select K is to set it equal to the square root of the number of data points in the training set.
- **Testing Multiple K Values**: You can try different K values and choose the one that works best on validation data.
- **Weighted Voting**: In cases where some neighbors are much closer than others, you can give them more weight in the voting process.



Training Data

Sepal Length	Sepal Width	Species
5.3	3.7	Setosa
5.1	3.8	Setosa
7.2	3.0	Virginica
5.4	3.4	Setosa
5.1	3.3	Setosa
5.4	3.9	Setosa
7.4	2.8	Virginica
6.1	2.8	Verscicolor
7.3	2.9	Virginica
6.0	2.7	Verscicolor
5.8	2.8	Virginica
6.3	2.3	Verscicolor
5.1	2.5	Verscicolor
6.3	2.5	Verscicolor
5.5	2.4	Verscicolor

Test Data

Sepal Length	Sepal Width	Species
5.2	3.1	?

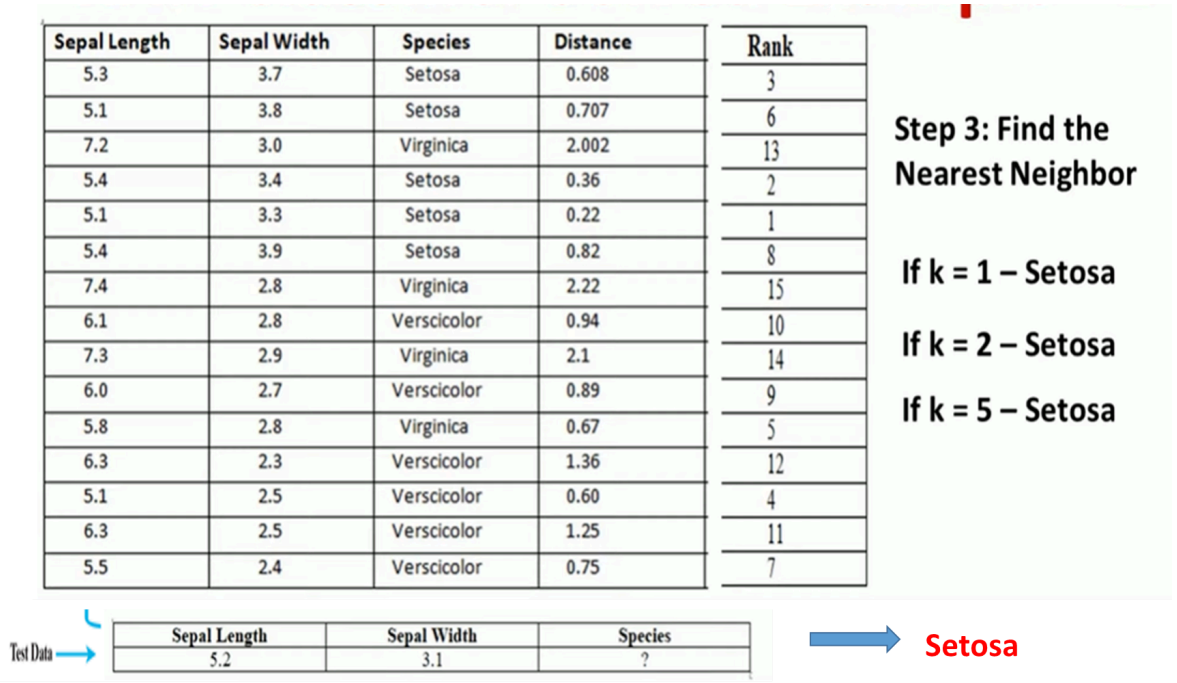
Step 1: Find Distance

$$\text{Distance (Sepal Length, Sepal Width)} = \sqrt{(x-a)^2 + (y-b)^2}$$

$$\text{Distance (Sepal Length, Sepal Width)} = \sqrt{(5.2-5.3)^2 + (3.1-3.7)^2}$$

$$\text{Distance (Sepal Length, Sepal Width)} = 0.608$$

Sepal Length	Sepal Width	Species	Distance
5.3	3.7	Setosa	0.608



Sepal Length	Sepal Width	Species	Distance	Rank
5.3	3.7	Setosa	0.608	3
5.1	3.8	Setosa	0.707	6
7.2	3.0	Virginica	2.002	13
5.4	3.4	Setosa	0.36	2
5.1	3.3	Setosa	0.22	1
5.4	3.9	Setosa	0.82	8
7.4	2.8	Virginica	2.22	15
6.1	2.8	Verscicolor	0.94	10
7.3	2.9	Virginica	2.1	14
6.0	2.7	Verscicolor	0.89	9
5.8	2.8	Virginica	0.67	5
6.3	2.3	Verscicolor	1.36	12
5.1	2.5	Verscicolor	0.60	4
6.3	2.5	Verscicolor	1.25	11
5.5	2.4	Verscicolor	0.75	7

Step 3: Find the Nearest Neighbor

If k = 1 – Setosa

If k = 2 – Setosa

If k = 5 – Setosa

Test Data

Sepal Length	Sepal Width	Species
5.2	3.1	?

→ **Setosa**

Problems :

1.https://www.youtube.com/watch?v=HZT0lxD5h6k&list=PLoE_I1sqGKLRXcbobCaczeu5y4sk6yKrL&index=2

2.https://www.youtube.com/watch?v=YndCelYQfgs&list=PLoE_I1sqGKLRXcbobCaczeu5y4sk6yKrL&index=5

3.https://www.youtube.com/watch?v=T0YkfWssHjk&list=PLoE_I1sqGKLRXcbobCaczeu5y4sk6yKrL&index=6

4.https://www.youtube.com/watch?v=WgSGdukowNA&list=PLoE_I1sqGKLRXcbobCaczeu5y4sk6yKrL&index=8

5.https://www.youtube.com/watch?v=kCNpLbCUo7g&list=PLoE_I1sqGKLRXcbobCaczeu5y4sk6yKrL&index=8

Logistic Regression is a supervised machine learning algorithm used for **classification** tasks, where the output is a categorical dependent variable. Unlike linear regression, which predicts continuous values, logistic regression predicts **probabilities** for a binary outcome (like Yes/No or 0/1). The predicted probabilities are mapped between 0 and 1 using a **sigmoid function**.

Key Features of Logistic Regression:

- **Binary Classification**: The output is categorical (e.g., Yes/No, 0/1).
- **Probabilistic Output**: Logistic regression doesn't directly predict 0 or 1, but rather probabilities that the outcome is 0 or 1.
- **Logistic Function (Sigmoid Function)**: This function maps predicted values to probabilities between 0 and 1, creating an "S-shaped" curve.
  Sigmoid function: $\text{Sigmoid function: } \frac{1}{1+e^{-z}}$



Types of Logistic Regression:

1. **Binomial**: The dependent variable has two possible types, e.g., 0 or 1.
2. **Multinomial**: More than two unordered categories, e.g., "cat", "dog", "rabbit".
3. **Ordinal**: More than two ordered categories, e.g., "low", "medium", "high".

Assumptions:

- The dependent variable must be **binary** (for binary logistic regression).
- Independent variables should not be highly correlated (low **multicollinearity**).
- Large sample sizes are preferred.

Logistic Regression Equation:

Logistic regression builds on the linear regression equation. It adjusts the linear output to a probability using the sigmoid function. The basic logistic regression equation is:

$$P(y=1|X) = \frac{1}{1+e^{-(\beta_0+\beta_1 X)}}$$

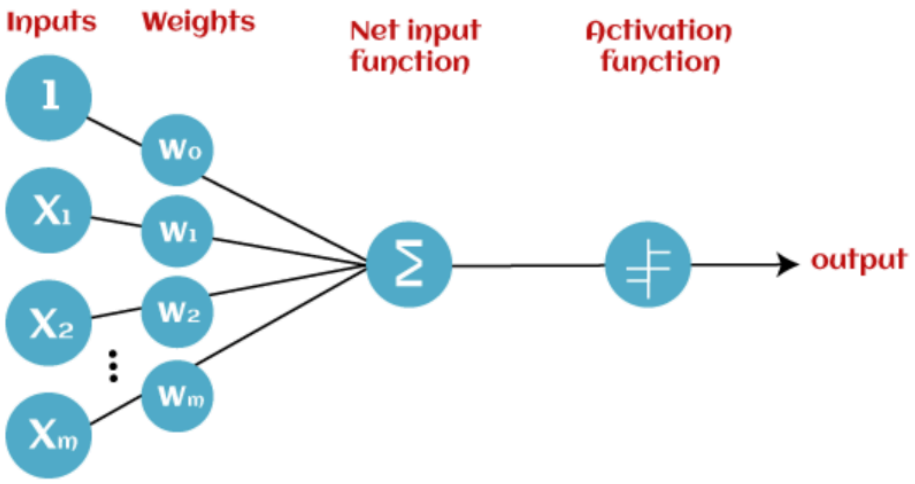
Where β_0 and β_1 are coefficients, and X represents the features.

Example Problems
1.https://www.youtube.com/watch?v=2C8IqOLO1os
2.https://www.youtube.com/watch?v=UCOm-LFKX9E

A **Perceptron** is the simplest type of artificial neural network, serving as the foundation for more complex neural networks. It is a **supervised learning algorithm** used for **binary classification**, which means it classifies input data into one of two categories (e.g., 0 or 1, Yes or No).

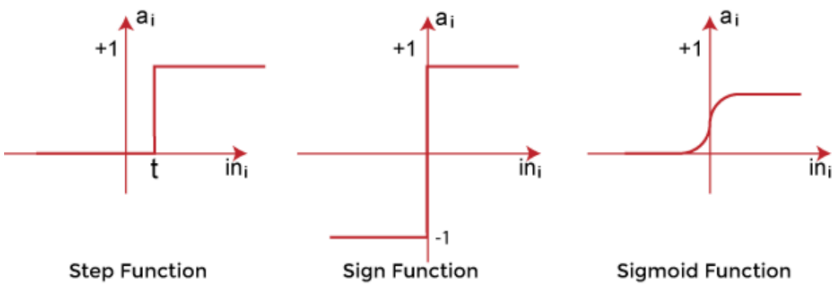
Binary classifiers can be considered as linear classifiers. In simple words, we can understand it as a classification algorithm that can predict linear predictor function in terms of weight and feature vectors.

- The **Perceptron** is a foundational algorithm in Machine Learning and Artificial Neural Networks.
- Invented by **Frank Rosenblatt** in the mid-20th century, it was designed for binary classification tasks.
- It is a **linear, supervised learning algorithm** used for classifying data into two categories.



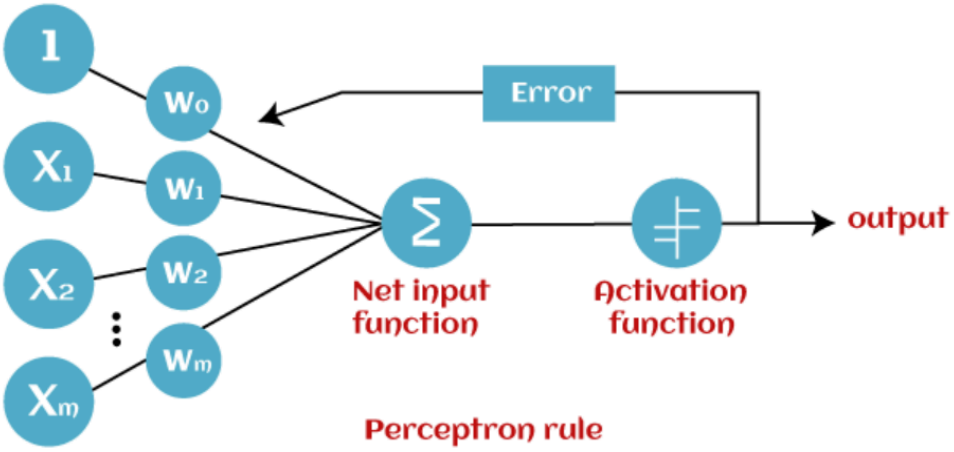
Key Concepts

- **Binary Classifier**:
  A function that determines whether an input belongs to one of two classes.
  Perceptrons create a **linear decision boundary** for classification tasks.
- **Components**:
  1. **Input Layer**: Accepts numerical input data.
  2. **Weights and Bias**:
     - **Weights**: Measure the strength of the connection between input and output.
     - **Bias**: Allows shifting of the decision boundary.
  3. **Net Sum**: Weighted sum of inputs and bias ($\sum wi \cdot xi+b$).
  4. **Activation Function**: Determines the output (e.g., Step, Sign, Sigmoid functions).



How It Works

1. **Step 1**: Compute the weighted sum: Net Sum=∑wi⋅xi+b
2. **Step 2**: Apply the activation function: Y=f(Net Sum)
    - Outputs binary values (e.g., 0 or 1, -1 or 1).



Key Features of a Perceptron:

1. **Inputs**: Takes multiple input features (e.g., x1,x2,x3,…)
2. **Weights**: Each input is associated with a weight that adjusts its influence on the output.
3. **Summation**: The perceptron computes a weighted sum of the inputs.
4. **Activation Function**: The sum is passed through an activation function (usually a step function) that outputs 0 or 1 based on a threshold.

Learning Process:

- **Training**: The perceptron adjusts its weights using **gradient descent** and the **perceptron learning rule** until it correctly classifies all training examples.
- **Output**: It predicts 0 or 1 depending on whether the weighted sum of inputs is below or above the threshold.

Limitations:

- Perceptrons can only solve **linearly separable** problems (e.g., classifying data that can be separated by a straight line).
- They struggle with more complex tasks, which is why multi-layer perceptrons (MLPs) were developed to solve nonlinear problems.

Applications: Basic binary classification tasks such as spam detection or image recognition (for simple patterns).

Single-Layer Perceptron (SLP)

A **Single-Layer Perceptron** is the simplest type of neural network. It is composed of a single layer of neurons (or nodes) that process inputs and produce outputs. It's used for solving **binary classification** problems when the data is **linearly separable**.

Structure

1. **Inputs (x1,x2,...,xn)**: Features of the data.
2. **Weights (w1,w2,...,wn)**: Values associated with each input, adjusting their importance.
3. **Bias (b)**: A constant added to the weighted sum to shift the decision boundary.
4. **Weighted Sum**: Computes $z = \sum_{i=1}^{n} w_i x_i + b.$
5. **Activation Function**: $$y = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

How It Works

1. **Forward Pass**:
   - Takes inputs, computes the weighted sum, and passes it through the step activation function to produce an output (0 or 1).
2. **Training**:
   - Uses the **Perceptron Learning Rule**:

$$w_i \leftarrow w_i + \eta \cdot (y_{true} - y_{pred}) \cdot x_i$$

$$b \leftarrow b + \eta \cdot (y_{true} - y_{pred})$$

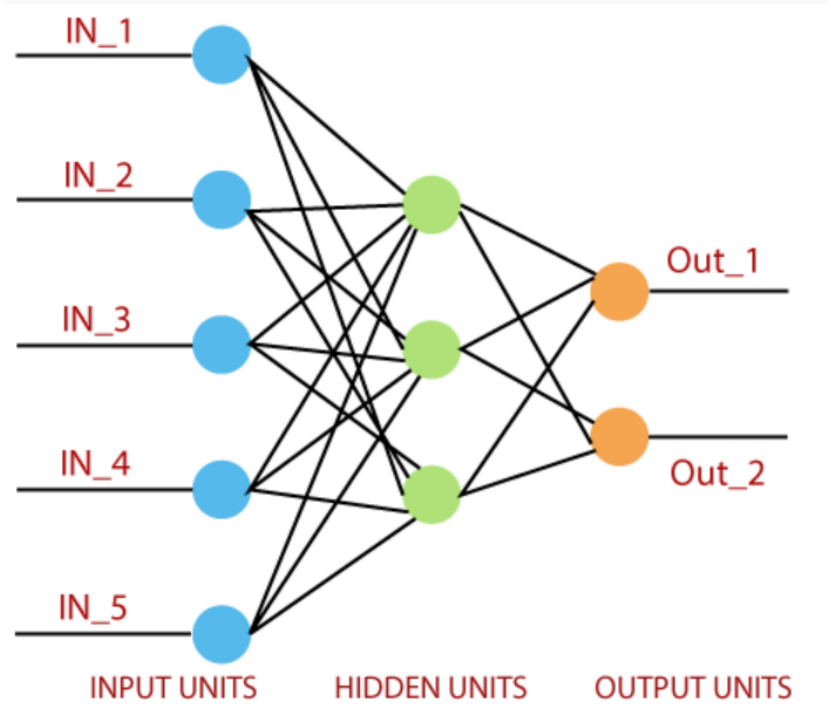
   - Where η is the learning rate.
   - Weights are updated iteratively until all training examples are correctly classified.

Applications of Single-Layer Perceptron

1. Implementing simple **logical gates** like AND, OR, and NOT.
2. Solving **binary classification problems** where data is linearly separable.

Limitations

1. **Linearity**:
   - Can only classify data that is linearly separable (e.g., NOT, AND, OR).
   - Cannot handle non-linear problems like XOR.
2. **Fixed Activation**:
   - Uses a step function, which is not differentiable, limiting flexibility.



Multi-Layer Perceptron (MLP)

https://www.youtube.com/watch?v=tTjcakAuHPI&list=PL4gu8xQu0_5JBO1FKRO5p20wc8DprlOgn&index=94

A **Multi-Layer Perceptron (MLP)** is a type of neural network that consists of an **input layer**, one or more **hidden layers**, and an **output layer**. It is capable of solving **non-linear problems**, making it more powerful than the single-layer perceptron.

Structure of MLP

1. **Input Layer**:
   - Accepts the input features (x1,x2,...,xn) from the dataset.
2. **Hidden Layers**:
   - One or more layers of neurons between the input and output layers.
   - Each neuron applies weights, biases, and a non-linear activation function (e.g., Sigmoid, ReLU).
   - These layers enable the MLP to learn complex relationships.
3. **Output Layer**:
   - Produces the final output (e.g., class probabilities or regression values).

How It Works

1. Forward Propagation:

- Data flows through the network:
  - Compute the weighted sum of inputs ($z=\sum wixi+b$) for each neuron.
  - Apply a non-linear **activation function** (e.g., Sigmoid, ReLU) to introduce non-linearity.
- The output of one layer becomes the input for the next layer.

2. Backward Propagation:

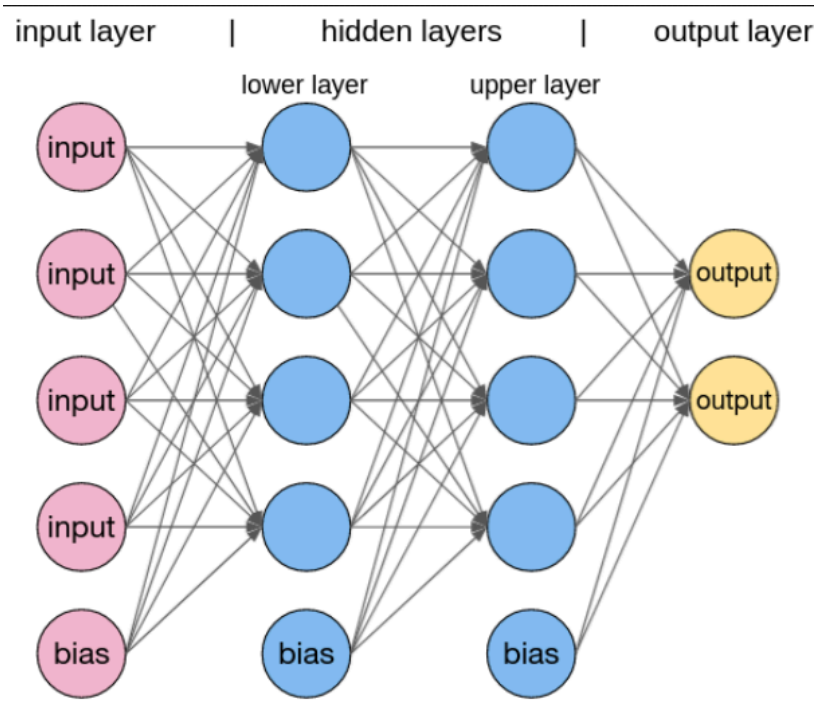
- Error (difference between predicted and actual output) is calculated using a loss function (e.g., Mean Squared Error, Cross-Entropy Loss).
- The error is propagated backward through the network using the **chain rule** to compute gradients.
- Weights are updated using **gradient descent** or its variants (e.g., SGD, Adam):

$$w \leftarrow w - \eta \cdot \frac{\partial L}{\partial w}$$ Where η is the learning rate, and L is the loss.

1.https://www.youtube.com/watch?v=tUoUdOdTkRw&list=PL4gu8xQu0_5JBO1FKRO5p20wc8DprlOgn&index=88

2.https://www.youtube.com/watch?v=n2L1J5JYgUk&list=PL4gu8xQu0_5JBO1FKRO5p20wc8DprlOgn&index=89

3.https://www.youtube.com/watch?v=AWhboi1aTxI&list=PL4gu8xQu0_5JBO1FKRO5p20wc8DprlOgn&index=90



Activation Functions in MLP

1. **Sigmoid**: Outputs values between 0 and 1, good for probabilities:

$$f(x) = \frac{1}{1 + e^{-x}}$$

2. **ReLU (Rectified Linear Unit)**: Outputs max(0,x) helps avoid vanishing gradients.
3. **Tanh**: Outputs values between -1 and 1, useful for symmetric outputs.

Key Features of MLP

1. **Non-Linearity**:
    - Hidden layers and non-linear activation functions allow MLPs to model complex relationships.
2. **Universal Approximation**:
    - An MLP with sufficient hidden neurons and layers can approximate any function.
3. **Deep Learning**:
    - Forms the basis of modern deep neural networks when extended with many hidden layers.

Applications of MLP

1. **Classification**:
    - Handwritten digit recognition (e.g., MNIST dataset).
    - Sentiment analysis (positive/negative classification).
2. **Regression**:
    - Predicting housing prices.
3. **Clustering and Feature Extraction**:
    - Learning representations of data for tasks like anomaly detection.
4. **Complex Tasks**:
    - Image recognition, speech recognition, and natural language processing (with deep extensions).

MLP Example: Solving XOR Problem

- XOR is not linearly separable, but MLP solves it using two hidden neurons:
    1. Hidden neurons learn to split the input space into non-linear regions.
    2. The output layer combines these regions to form the XOR logic.

Advantages

1. Can handle non-linear problems.
2. Flexible and can adapt to a wide range of tasks.
3. Enables deeper networks for more complex tasks.

Limitations

1. Computationally expensive for large datasets.
2. Susceptible to overfitting without proper regularization.
3. Requires a lot of labeled data for training.

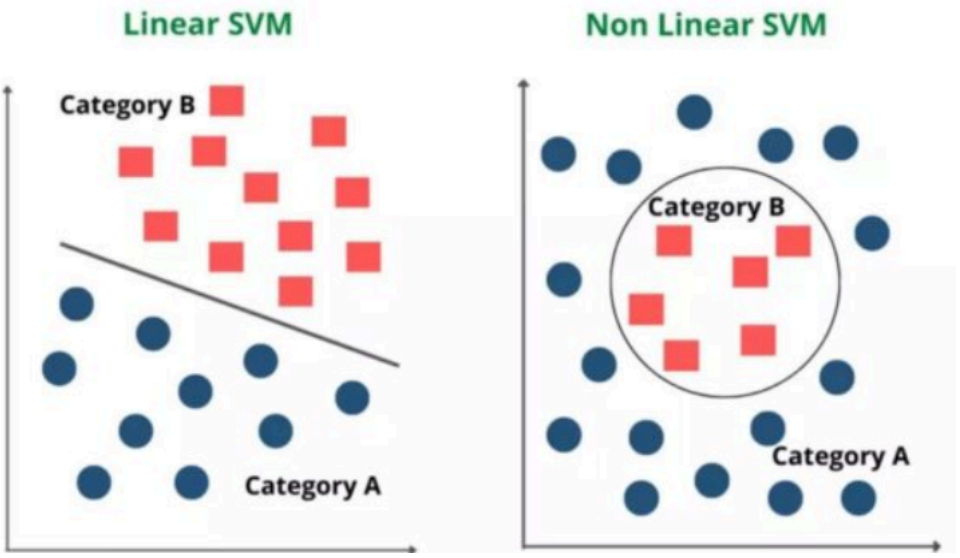
perceptron:

- https://www.youtube.com/watch?v=ItkSCYzSD34&list=PL4gu8xQu0_5JBO1FKRO5p20wc8DprlOgn&index=91
- https://www.youtube.com/watch?v=du5fyS44DR8&list=PL4gu8xQu0_5JBO1FKRO5p20wc8DprlOgn&index=76
- https://www.youtube.com/watch?v=zmIzNBMsQYQ&list=PL4gu8xQu0_5JBO1FKRO5p20wc8DprlOgn&index=77
- https://www.youtube.com/watch?v=uxmNDNb0u9A&list=PL4gu8xQu0_5JBO1FKRO5p20wc8DprlOgn&index=78
- https://www.youtube.com/watch?v=dM_8Y41EgsY&list=PL4gu8xQu0_5JBO1FKRO5p20wc8DprlOgn&index=7

Support Vector Machines (SVM) is a powerful supervised machine learning algorithm used for classification and regression tasks, though it is more commonly applied to classification problems. SVM works by finding a hyperplane that best divides a dataset into different classes.

The goal of SVM is to find a decision boundary (or hyperplane) that maximizes the margin between data points of different classes. The margin is the distance between the hyperplane and the closest data points from either class, known as **support vectors**. The idea is that the wider the margin, the better the generalization ability of the model.

- **Linear SVM**: If the data is linearly separable, a straight line (in 2D), a plane (in 3D), or a hyperplane (in higher dimensions) can separate the two classes.
- **Non-linear SVM**: If the data is not linearly separable, SVM can still perform well by transforming the data into a higher-dimensional space where a linear hyperplane can be used for separation (through the **kernel trick**).



Margin:The margin is the distance between the decision boundary (hyperplane) and the closest data points from either class. SVM aims to maximize this margin to improve the model's generalization ability. A larger margin reduces the risk of overfitting.

Support Vectors:Support vectors are the data points that are closest to the decision boundary (hyperplane). These points are critical in defining the position and orientation of the hyperplane. They are the most important data points for constructing the model.

Hyperplane:A hyperplane is a decision boundary that separates data points of different classes. In 2D, it is a line; in 3D, a plane; and in higher dimensions, it becomes a hyperplane.

Linear SVM

When the data is linearly separable, SVM tries to find the hyperplane that best separates the two classes. The hyperplane can be represented as:

$$w \cdot x + b = 0$$

Where:

- w is the weight vector (normal to the hyperplane).
- x is the feature vector.
- b is the bias term (distance of the hyperplane from the origin).

Objective:

- Maximize the margin, which is defined as the distance from the hyperplane to the nearest data points (support vectors).
- The distance of a data point xi from the hyperplane is given by:

$$\text{Distance} = \frac{|w \cdot x_i + b|}{\|w\|}$$

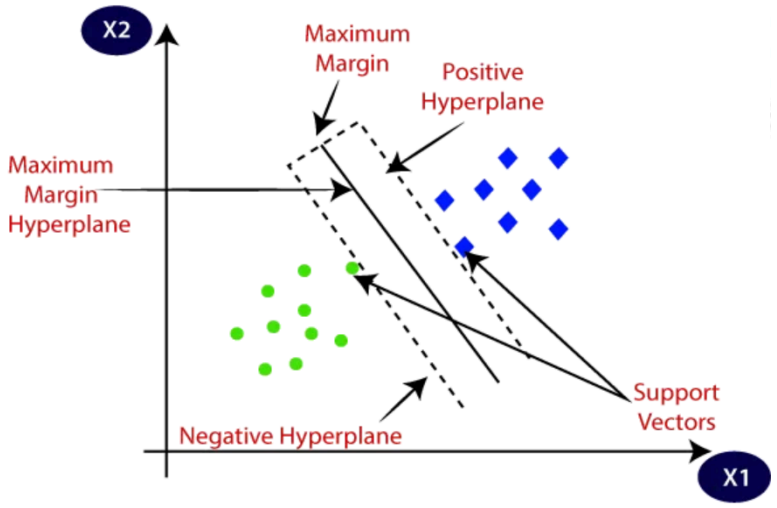
Maximizing the margin is equivalent to minimizing the norm $\|w\|$, or equivalently, solving the following optimization problem:

$$\min \frac{1}{2}\|w\|^2$$

Subject to the constraints:

$$y_i(w \cdot x_i + b) \geq 1 \quad \text{for all} \quad i$$

Where y_i is the class label (+1 or -1) of the ith data point.



Linear SVM

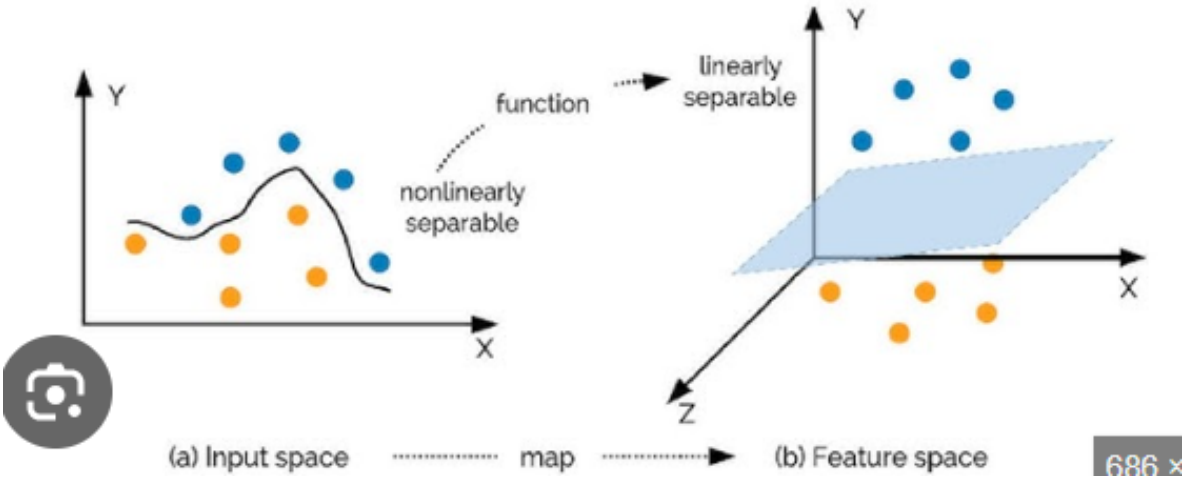
Non-linear SVM and Kernel Trick

In many real-world problems, the data is not linearly separable. In these cases, SVM uses the **kernel trick** to map the data into a higher-dimensional feature space where it becomes possible to find a linear hyperplane that separates the classes.

The kernel trick involves using a function $\phi(x)$ that implicitly maps the data into a higher-dimensional space without having to compute the transformation explicitly. Some commonly used kernels are:

- **Linear Kernel**: $K(x, x') = x \cdot x'$
- **Polynomial Kernel**: $K(x, x') = (x \cdot x' + c)^d$
- **Radial Basis Function (RBF) Kernel**: $K(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right)$
- **Sigmoid Kernel**: $K(x, x') = \tanh(\alpha x \cdot x' + c)$

Using the kernel trick, the SVM's decision boundary in the higher-dimensional space is a hyperplane, which is equivalent to a non-linear decision boundary in the original input space.



SVM for Multi-class Classification

SVM is inherently a binary classifier, but it can be extended to multi-class classification problems using strategies like:

- **One-vs-One (OvO)**: Create a classifier for every pair of classes. If there are k classes, then the total number of classifiers is k(k−1)/2
- **One-vs-Rest (OvR)**: Create a classifier for each class, where each classifier distinguishes one class from all other classes.

Advantages of SVM

- **Effective in high-dimensional spaces**: SVM works well in spaces with more features than samples.
- **Memory efficient**: SVM uses a subset of training points (support vectors) in the decision function.
- **Robust to overfitting**: Especially in high-dimensional space, given the use of the regularization parameter.

Disadvantages of SVM

- **Computationally intensive**: SVM can be slow for large datasets because of the quadratic optimization problem it involves.
- **Choice of kernel and hyperparameters**: The performance of SVM is highly dependent on the correct selection of kernel, C, and other hyperparameters.

Applications:

- **Text Classification**: Used in spam detection, sentiment analysis, and document categorization.
- **Image Classification & Object Recognition**: Applied in face recognition, handwriting recognition, and medical image analysis (e.g., tumor detection).
- **Biological & Biomedical**: Used in gene expression data analysis, cancer detection, and protein classification.
- **Finance**: Applied in credit scoring, fraud detection, and stock market predictions.
- **Speech Recognition**: Used for classifying speech data in voice recognition systems.
- **Anomaly Detection**: Detects outliers or abnormal behavior in various fields like network security and manufacturing.

Soft Margin Svm:

In real-world scenarios, the data is often not perfectly separable, which can result in overfitting if we strictly enforce a perfect separation. To handle this, SVM introduces the concept of a **soft margin**, which allows some misclassification of data points.

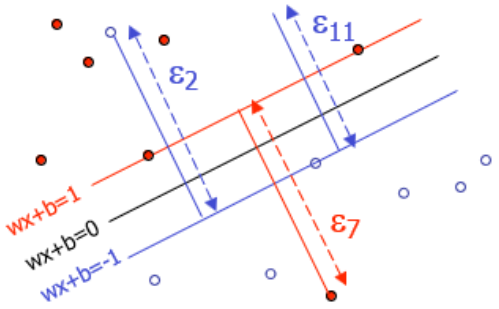
The objective function for soft margin SVM becomes:

$$\min \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i$$

Where:

- ξ_i is a slack variable that measures the degree of misclassification for the ith data point.
- C is a regularization parameter that controls the trade-off between maximizing the margin and minimizing the classification error. A large C gives more importance to minimizing misclassification, while a smaller C allows for a larger margin at the cost of more errors.

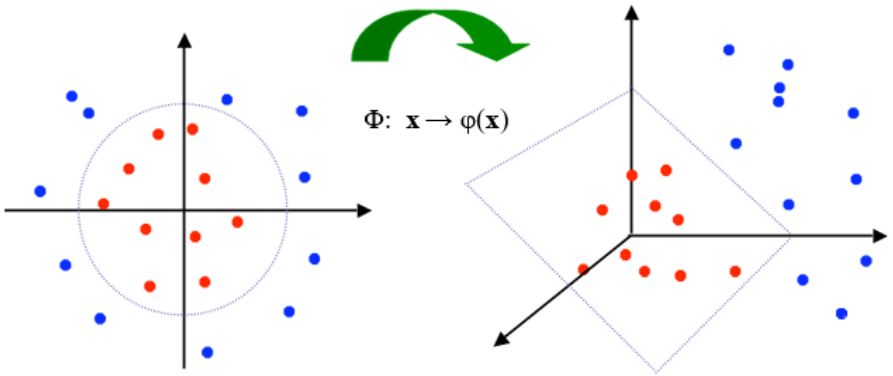
Slack variables ε_i **can be added to allow misclassification of difficult or noisy examples.**



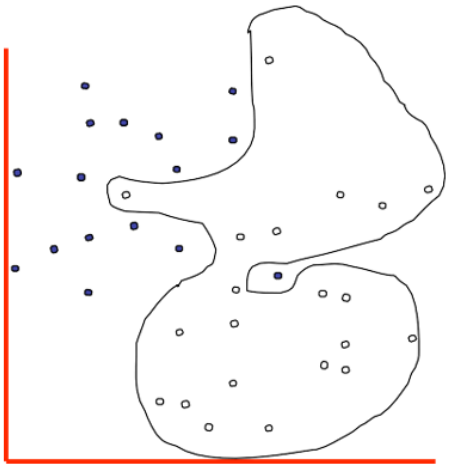
Minimize:

$$\frac{1}{2}\mathbf{w}.\mathbf{w} + C\sum_{k=1}^{R}{}''_{k}$$

Non-linear SVMs: Feature spaces



The original input space can always be mapped to some higher-dimensional feature space where the training set is separable:



Hard Margin: requires that all data points are classified correctly

What if the training set is noisy?

- Solution: use very powerful kernels

OVERFITTING!

Playlist(for Problems related to SVM):
https://www.youtube.com/playlist?list=PL4gu8xQu0_5JdEqxU_FcY2x2aPte_BzF5

Metrics for SVM

To evaluate the performance of an SVM model, common metrics used are:

- **Accuracy**: Proportion of correct predictions.
- **Precision**: $\frac{TP}{TP+FP}$ – The proportion of true positives among all positive predictions.
- **Recall (Sensitivity)**: $\frac{TP}{TP+FN}$ – The proportion of actual positives correctly identified.
- **F1-Score**: Harmonic mean of precision and recall: $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$
- **ROC-AUC**: Measures the area under the Receiver Operating Characteristic curve, which shows the trade-off between true positive rate and false positive rate.

Error Correction and Hyperparameter Tuning

SVM models can be sensitive to hyperparameters, and improper selection can lead to poor performance. Key hyperparameters include:

- **C**: Regularization parameter that controls the trade-off between margin size and classification error.
- **Kernel**: Type of kernel to use (e.g., linear, RBF, polynomial).
- **Gamma** (for RBF kernel): A parameter that controls the influence of a single training example on the decision boundary.

Techniques like **grid search** or **random search** can be used to tune these hyperparameters and find the optimal configuration. Cross-validation is also essential to evaluate the model's generalization ability.